

CHAPTER 15: RECURSION

COSC 221 (Computer Science B)

Contents

- Introduction
- Base Case and Recursive Case
- Types of Recursion in Python
- Recursion vs Iteration
- Advantages of using recursion
- Disadvantages of using recursion
- Applications

Objective

- To understand the concept of **Recursion**.
- To be able to **think** recursively.
- To compare it with **iteration**.
- To be able to solve problems using **recursion**.
- To understand some pros and cons of **recursion**.
- To be able to **apply recursion** to solve some challenges.

15.1. Introduction with Factorial Function

To understand
Recursion, you
just need to
understand
Recursion.



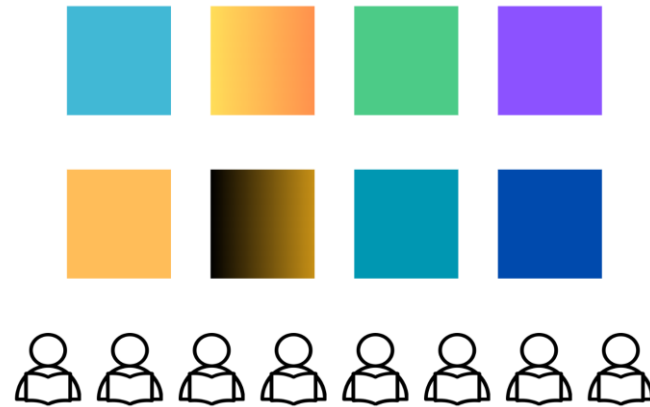
Introduction

- Consider an example:

The number of arrangements of n objects into n places is denoted by $n!$ (n factorial) and is defined by:

$$n! = n \times (n - 1) \times \cdots \times 1$$

with $0! = 1$.



- Write a **Python function** to compute the number of number of ways for n students to sit at n tables, which is $n!$

Introduction

- We can do:

```
def factorial(n):
```

```
    if n == 1 or n == 0:
```

```
        return 1
```

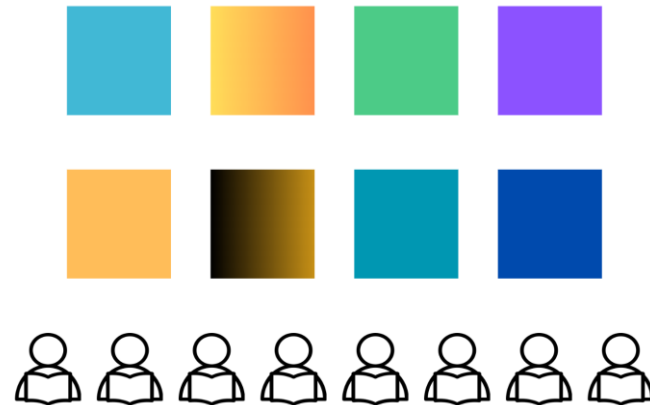
```
    else:
```

```
        x = 1
```

```
        for i in range(2,n+1):
```

```
            x *= i
```

```
        return x
```



- Observation:

- $n! = n \times (n - 1) \times \cdots \times 1 = n \times (n - 1)!$
- If you treat $n!$ as some expression $f(n)$ then it satisfies recurrence relation:

$$f(n) = n \times f(n - 1).$$

Introduction

- Using this idea: $n! = n \times (n - 1)!$ we do something like:

```
def factorial(n):  
    return n * factorial(n-1)
```

- Python** allows you to call the function that is being defined!



- However, there's a problem here! You don't know where to stop!
- We need a **stopping condition** called, **Base case**.
- For our factorial function, it's when $n = 1$ or $n = 0$.

Introduction

- So we have a complete function for $n! = n \times (n-1)!$

```
def factorial(n):
```

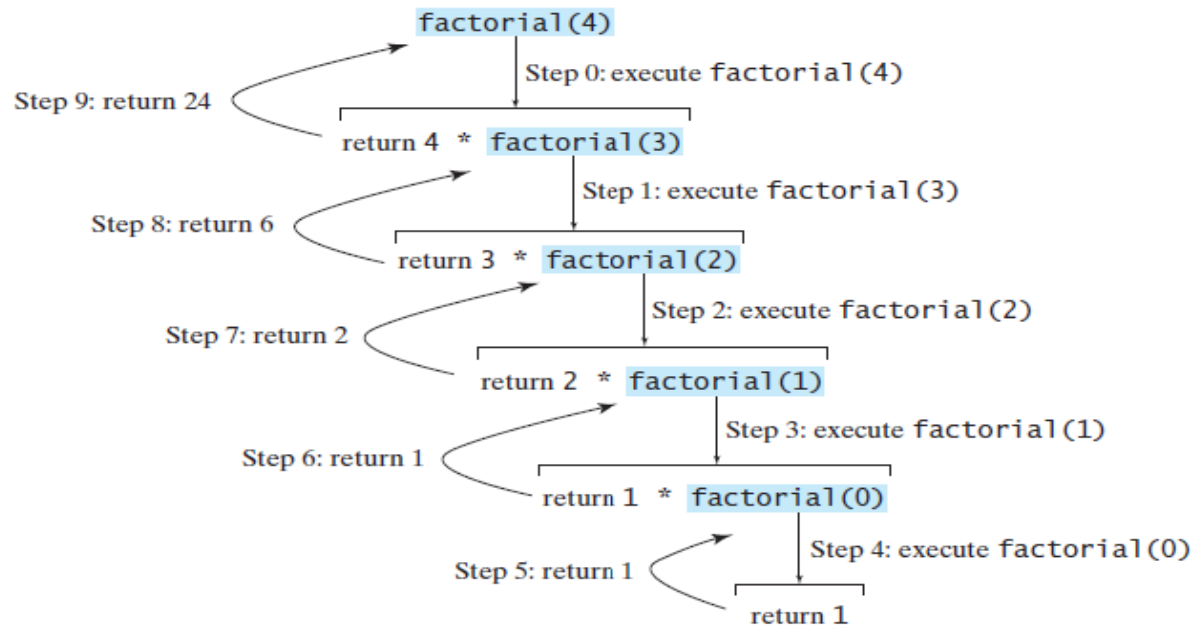
```
    if n in {0,1}:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

Base case



Introduction

- **Key: Recursion can often be used when/with**
 - **if-else** statements that distinguish different cases.
 - There **exists at least one condition (base case)** to stop the recursion.
 - Every recursive call reduces the original problem closer to **base case**.
 - The problems can often be written as **recurrence relation**.



Introduction

- **Key: Recursion can often be used when/with**
 - **if-else** statements that distinguish different cases.
 - There **exists at least one condition (base case)** to stop the recursion.
 - Every recursive call reduces the original problem closer to **base case**.
 - The problems can often be written as **recurrence relation**.
- **Challenge1:** Compute $n!$ for $n = 0, 4, 8, 10$ using both functions: recursion and iteration.
- **Challenge2:** Write a recursive function to compute S_n for $n = 10, 100, 200, 300$ with

$$S_n = 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \dots + \frac{(-1)^{n-1}}{n}.$$

Can you compute S_{10000} ? **Hint:** Write relation between S_n and S_{n-1} .

15.2. Fibonacci Numbers

Fibonacci Numbers

- **Fibonacci sequence/numbers** was used to study the growth of rabbit population. It's named after an Italian mathematician.
- The numbers are: 1, 1, 2, 3, 5, 8, 13, 21, 34...
- If $F_0 = F_1 = 1$ are the first two terms of this numbers, then

$$F_n = F_{n-1} + F_{n-2}$$

for $n \geq 2$.



- They look simple but appear everywhere including in **nature** and **algorithm analysis**.
- **Ex:** If the population of rabbits in a forest at n -th month is given by F_n , write a recursive function to compute the population of the rabbits at each month of the first 15 months.

Fibonacci Numbers

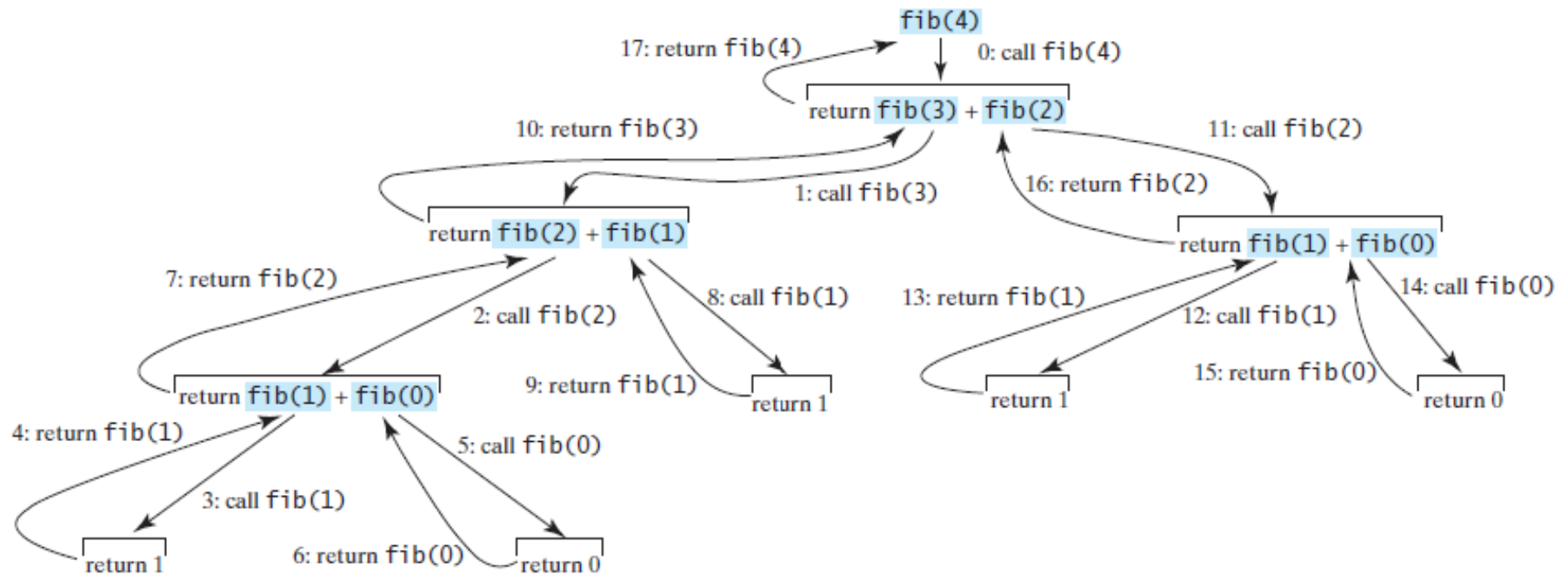
```
1 def main():
2     index = eval(input("Enter an index for a Fibonacci number: "))
3     # Find and display the Fibonacci number
4     print("The Fibonacci number at index", index, "is", fib(index))
5
6 # The function for finding the Fibonacci number
7 def fib(index):
8     if index == 0: # Base case
9         return 0
10    elif index == 1: # Base case
11        return 1
12    else: # Reduction and recursive calls
13        return fib(index - 1) + fib(index - 2)
14
15 main() # Call the main function
```



- **Ex:** If the population of rabbits in a forest at n -th month is given by F_n , write a recursive function to compute the population of the rabbits at each month of the first 15 months.

Fibonacci Numbers

- Fibonacci calls with $n = 4$:



- Number of recursive calls in $F(n)$

n	2	3	4	10	20	30	40	50
# of calls	3	5	9	177	21891	2692537	331160281	2075316483

Fibonacci Numbers

- Recursion is not an efficient way to compute Fibonacci numbers, but it is an easy way to understand how recursion works.
- Thinking recursively is fun. It's also healthy if you drink water recursively during the day:

```
def drinkWater(bottle):  
    if not bottle.isEmpty():  
        bottle.takeOneSip() # take a sip, water decreases a bit  
        drinkWater(bottle)
```

15.3. Palindrome Problem

Palindrome Problem

1. Palindrome problem with recursive

- A string is a **palindrome** if it reads the same from the left and from the right.
- Ex: “Alexela” is palindrome but “Alexi” is not.
- **Challenge:** write a recursive function **isPalindrome(s)** to check if a string **s** is palindrome or not.

Palindrome Problem

1. Palindrome problem with recursive

- A string is a **palindrome** if it reads the same from the left and from the right.
- Ex: “Alexela” is palindrome but “Alexi” is not.
- **Challenge:** write a recursive function **isPalindrome(s)** to check if a string **s** is palindrome or not.

```
1 def isPalindrome(s):
2     if len(s) <= 1: # Base case
3         return True
4     elif s[0] != s[len(s) - 1]: # Base case
5         return False
6     else:
7         return isPalindrome(s[1 : len(s) - 1])
8
```

Palindrome Problem

2. Recursive helper functions

- **isPalindrome** above is not efficient as it creates new string at every call.
- To avoid this, we can fix the string **s** and use another helper function called **Recursive Helper Function**.

```
1 def isPalindrome(s):  
2     return isPalindromeHelper(s, 0, len(s) - 1)  
3  
4 def isPalindromeHelper(s, low, high):  
5     if high <= low: # Base case  
6         return True  
7     elif s[low] != s[high]: # Base case  
8         return False  
9     else:  
10        return isPalindromeHelper(s, low + 1, high - 1)
```

Palindrome Problem

Demo

15.4. Selection Sort

Selection Sort

- Selection sorting is based on two ideas:
 - Find the **smallest element** of a list then **swap it with the first element** of the list.
 - **Ignore the first element** and continue **sorting the shorter list**.
- Ex: sort $L = [3, 2, 5, 1, 2, 6, 3]$
 - Find the smallest element = 1 then swap with 3, then $L = [1, 2, 5, 3, 2, 6, 3]$.
 - Do the same with smaller list $L1 = [2, 5, 3, 2, 6, 3]$.
 - Repeat the same process: $L2 = [5, 3, 2, 6, 3] \rightarrow [2, 3, 5, 6, 3]$.
 - $L3 = [3, 5, 6, 3] \rightarrow L4 = [5, 6, 3] \rightarrow [3, 6, 5] \rightarrow L5 = [6, 5] \rightarrow [5, 6]$.

Selection Sort

```
1 def sort(lst):
2     sortHelper(lst, 0, len(lst) - 1) # Sort the entire list
3
4 def sortHelper(lst, low, high):
5     if low < high:
6         # Find the smallest element and its index in lst[low .. high]
7         indexOfMin = low
8         min = lst[low]
9         for i in range(low + 1, high + 1):
10             if lst[i] < min:
11                 min = lst[i]
12                 indexOfMin = i
13
14         # Swap the smallest in lst[low .. high] with lst[low]
15         lst[indexOfMin] = lst[low]
16         lst[low] = min
17
18         # Sort the remaining lst[low+1 .. high]
19         sortHelper(lst, low + 1, high)
```

- Generate a few random lists, and test out your function.

15.5. Binary Search

Binary Search

- Just like when searching for a letter in a pile of letters ordered by names. We often start at the middle then move up or down according to the name.
- Binary search looks for a key in a sorted list by breaking it down by half and continue searching according to the position of the key relative to the break point.
- Ex: I want to search for 7 in a sorted list L of size 10.
 - I break L into L1 and L2 where $L2[0] > 7$.
 - Then I look for 7 in L1 only.

Binary Search

```
1 def recursiveBinarySearch(lst, key):
2     low = 0
3     high = len(lst) - 1
4     return recursiveBinarySearchHelper(lst, key, low, high)
5
6 def recursiveBinarySearchHelper(lst, key, low, high):
7     if low > high: # The list has been exhausted without a match
8         return -low - 1
9
10    mid = (low + high) // 2
11    if key < lst[mid]:
12        return recursiveBinarySearchHelper(lst, key, low, mid - 1)
13    elif key == lst[mid]:
14        return mid
15    else:
16        return recursiveBinarySearchHelper(lst, key, mid + 1, high)
```

helper function
base case

recursive call
base case

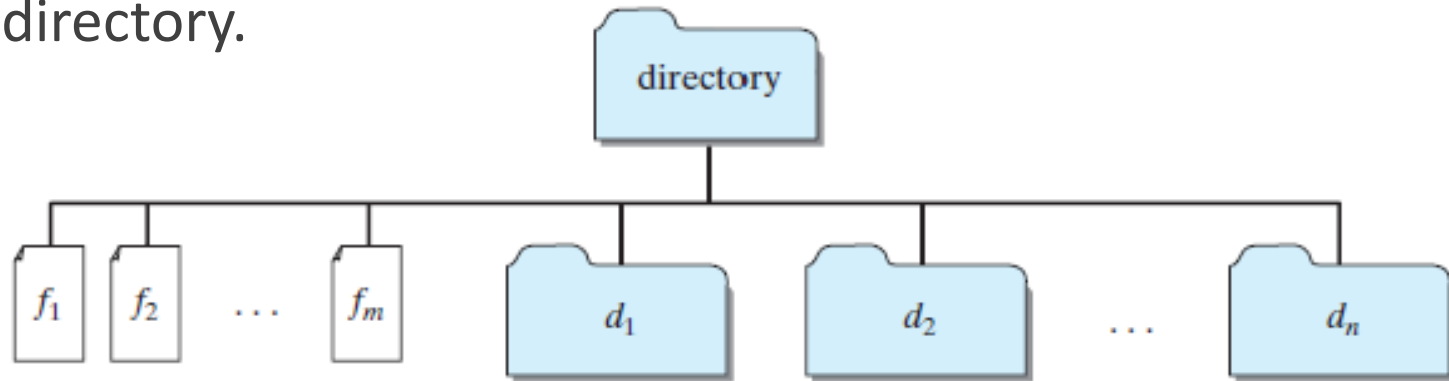
recursive call

- Generate a few random lists, sort them, then search for a **key** in each list.

15.6. Finding Directory Size

Finding Directory Size

- The directory size is the sum of all file size within that directory.



- Finding directory size may be hard without recursion. To solve this, we need:
 - `os.path.isfile(s)`**, which returns True if `s` is a filename.
 - `os.path.getsize(filename)`**, which returns the size of the file.
 - `os.listdir(directory)`**, which returns a list of the subdirectories and files under the directory.

Finding Directory Size

```
1 import os
2
3 def main():
4     # Prompt the user to enter a directory or a file
5     path = input("Enter a directory or a file: ").strip()
6
7     # Display the size
8     try:
9         print(getSize(path), "bytes")
10    except:
11        print("Directory or file does not exist")
12
13 def getSize(path):
14     size = 0 # Store the total size of all files
15
16     if not os.path.isfile(path):
17         lst = os.listdir(path) # All files and subdirectories
18         for subdirectory in lst:
19             size += getSize(path + "\\" + subdirectory)
20     else: # Base case, it is a file
21         size += os.path.getsize(path) # Accumulate file size
22
23     return size
```

invoke function

getSize function

is directory?
all subitems

recursive call

base case

- Test the function with your directory. Try non-existing directory as well.

15.7. Recursion vs Iteration

Recursion vs Iteration

- Iteration often requires body structure and number of iterations while recursion does not.
- Recursion consumes more time and memory in general.
- Any tasks can be solved recursively generally can be solved non-recursively with iteration?
- Why still need recursion? Because it sometimes can simplify tasks and easier to code, such as the directory size problem.
- Which method to use depends on the nature of the problem.
- Iteration often be more efficient than recursive if the tasks can be easily worked out using iteration.

Q & A

End of session



Thank you !!!

